YAMI: A Robot-Native, SLAM-Free Handheld YAM Interface

Atharva Kshirsagar
Independent Researcher
k7agar@gmail.com

Vrushtee Gaikwad
Independent Researcher
vrushteegaikwad@gmail.com

Tirth Gada
Independent Researcher
gadatirth45@gmail.com

Abstract: The I2RT YAM is the arm behind several recent open robot-learning efforts (ABC, MolmoAct 2, RoboTTT), and their reference rigs share one wrist camera: the RealSense D405. We build YAMI (YAM Interface), a UMI-style handheld gripper, from the same camera, the YAM's own fingers and kinematic model, 3D-printed parts and a RealSense T265, so demonstrations come out in the camera and tool frame the arm uses. Pose comes from the T265's onboard visual-inertial odometry, so there is no offline SLAM pass and tracking failures are visible while the demonstration is being recorded. We report the quality of 61 recorded demonstrations before any training and ACT policies trained on them that run on the YAM: with 44 training demonstrations the arm completes a full pick-and-place in 6 of 10 trials (8 of 10 grasps), against 1 of 10 grasps with 22. Checking with a learned workspace model whether the arm can follow what the hand did shows that, anchored at the arm's deployment start pose, only 83% of training frames are reachable, the misses cluster at the grasp, and a 10° tilt retarget raises this to 93% without changing the wrist images.

Keywords: Handheld data collection, Imitation learning, Robot-native demonstrations

1 Introduction

Handheld interfaces such as UMI [1] and FastUMI [2] decouple data collection from the robot, but the data must still be mapped onto the arm that runs the policy. UMI recovers pose offline with ORB-SLAM3 [3] on GoPro video: one mapping pass per scene, and tracking failures found only after the session.

The YAM has become a common target arm. ABC [4] (3,553 h, 134,806 bimanual episodes) and MolmoAct 2 [5] (over 700 h) collect on YAM stations, and RoboTTT [6] deploys on one. All three use D405 wrist cameras, and I2RT's ABC Box ships with D405 mounts for both wrists and the overhead view [7]. A handheld for this ecosystem can therefore reuse the arm's own camera, fingers and kinematics. We claim: **(1) robot-native, low-cost hardware**, where the only part YAMI adds to a YAM rig is a T265; and **(2) SLAM-free tracking**, with onboard VIO on the image clock and a confidence gate at record time. We report demonstration quality before training, policies on the arm, and an audit of whether the arm can do what the hand did.

2 System

Hardware. YAMI carries the D405 model used on the YAM wrist (640×480, 30 fps), the linear rails, carriages and fingertips of a YAM gripper, two 10 mm ArUco markers on the fingers for jaw

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

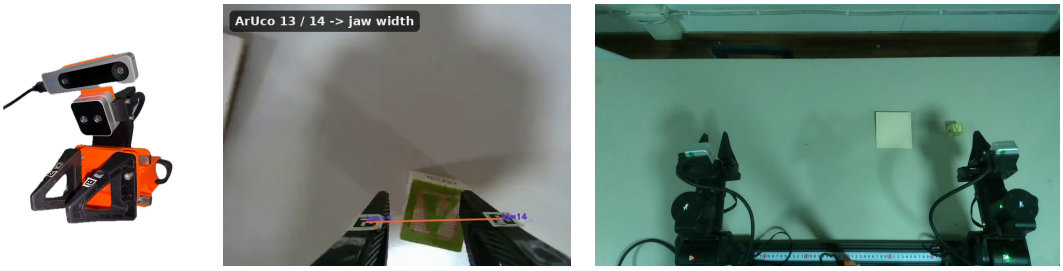


Figure 1: Left: YAMI, a T265 (top) and a D405 on a printed body with the rails, carriages and fingertips of a YAM gripper, joined by a printed rack and pinion. Middle: YAMI's wrist view; the finger markers give jaw width in the same image. Right: the deployment rig from the overhead D435 (the policy uses only the right arm's wrist D405).

Table 1: Hardware each interface adds for data collection (USD, Oct. 2026). Parts that ship with the YAM rig and printed parts count as zero; UMI's printed gripper ($73) is shown separately so the comparison is like-for-like.

Interface	What is added	Added cost
YAMI	T265 (discontinued; resale). D405 and gripper parts ship with the rig; body printed	**199–425**
UMI [1]	GoPro, mounts, mirrors ($298) + printed gripper ($73)	**298** (371)
FastUMI [2]	GoPro + T265, estimate [8]	**650–900**

width, and a T265 (Fig. 1). Its pose is expressed at the YAM tool frame, so the arm's model and IK apply to demonstrations unchanged.

Cost. Table 1 counts only what each interface adds. For YAMI that is the T265 ($199 at launch, $339–425 from resellers now), against $298 of GoPro hardware for UMI and an estimated $650–900 for FastUMI. The robot side needs nothing new: the arm already carries the D405 and the gripper YAMI copies.

Pose, width and recording. The T265 streams 6-DoF pose with a 0–3 confidence at 200 Hz on the same SDK clock as the D405, so each image is matched to pose by interpolation, with no offline synchronization or mapping pass. Jaw width comes from the finger markers in the same image, mapped to millimetres by a four-point ruler calibration over 0–53 mm; widths outside that range are rejected. An episode starts only after one second of confidence-3 pose and a valid width. A persistent session writes LeRobot v3 datasets with wrist video, full-rate pose and IMU, and lossless depth.

Robot side: Karma. On the arm we run Karma [9], an open-source control stack for bimanual YAM and SO100/SO101 arms. YAMI uses its YAM MJCF model and IK, both to check demonstrations against the arm (Sec. 5) and to turn policy poses into joint targets. It also uses Karma's motor runtime to power up, stream and park the arm, with state-freshness and joint-limit guards; its bounded executor for streaming action chunks; its camera capture for the arm's wrist D405; and its LeRobot writer for hardware replays. YAMI's handheld recording runs without Karma.

3 Demonstration quality

We recorded 61 pick-and-place demonstrations (green block onto a board) in three sessions and kept 45 (15,912 frames, 8.8 min; Table 2). In the fully audited session (25 episodes), all 63,363 pose samples have confidence 3, the largest pose gap is 5.1 ms, and 2 of 9,480 RGB frames were dropped. The failure modes are jaw-width dropouts and T265 relocalization jumps (steps above 67 mm between frames). Width coverage rose from 53% to 78–86% once the start gate became compulsory.

Replay on hardware. One YAMI demonstration replayed on the real YAM through the deployment planner reproduces the recorded path to 0.4 mm median (1.8 mm p95) in command, and the arm tracks it to 14 mm median with a 15 mm downward bias, with no conversion between hand and arm frames.

Table 2: Demonstration quality per session. "Width seen": kept frames with both finger markers detected; the rest are interpolated across short gaps.

	S1	S2	S3	All
Recorded / kept	26 / 16	10 / 7	25 / 22	61 / 45
Dropped: by request / width gap >2 s / pose jumps	4 / 6 / 0	2 / 1 / 0	0 / 1 / 2	6 / 8 / 2
Kept episodes trimmed at a jump	5	0	2	7
Width seen (frames)	52.7%	86.1%	78.1%	72.2%

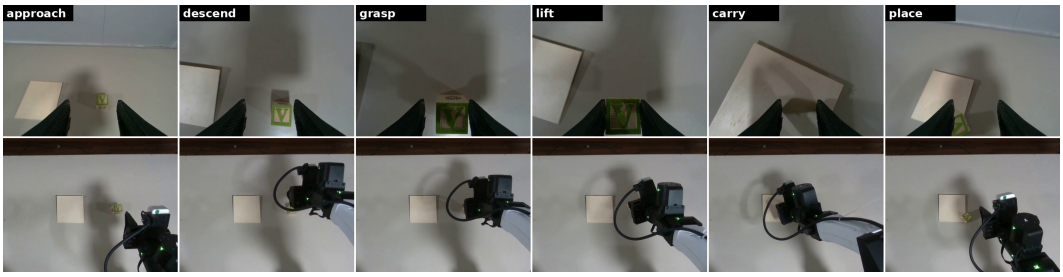


Figure 2: The 44-episode policy on the YAM. Top: the arm's wrist D405, the policy's only input. Bottom: overhead view (not an input).

4 Policies on the arm

Each 50-step action window becomes poses relative to the current pose, $T_{\text{now}}^{-1}T_k$, as in UMI, plus absolute jaw width. We train ACT [10] with a fine-tuned CLIP [11] ViT-B/16 backbone. At deployment the arm's wrist D405 replaces YAMI's, and relative chunks are composed with the measured pose, solved by IK and streamed through Karma.

A trial succeeds if the arm grasps the block, carries it to the board and places it without dropping it; the block starts anywhere in a 6 cm square (Fig. 2). The 44-episode model reaches the block in 10/10 trials, grasps it in 8/10 and places it in **6/10** (95% Wilson interval 31–83%); in the other two grasps it releases the block too high. The 22-episode model reaches every time but grasps only 1/10 and places 0/10. On a held-out episode, mean position error falls from 73.1 mm (13 episodes) and 57.7 mm (22) to 36.7 mm (44), and width error from 3.6 to 1.5 mm. The models also differ in recipe (the 44-episode model adds augmentation and jump trimming), so this is not a controlled scaling curve. Image to arm in place takes about 0.4 s, with 151–171 ms of CPU inference.

5 Can the arm do what the hand did?

A hand has none of the arm's joint limits; where the arm cannot follow, the policy trains on actions it will never execute. Because YAMI shares the arm's model, each demonstration can be checked before training. On a separate 10-episode session (6,024 frames) the stock IK fails on 14% of frames at 10 mm / 5°; a full 6-DoF QP step with the mink differential IK solver [12] recovers some of these, and no configuration reaches the rest.

Learned workspace. We distil converged multi-start IK into a 4×256 MLP on position and 6-D rotation [13], trained on 300k labelled poses (5 µs per pose). It agrees with full IK on 99.8% of block frames, 97.9% of an unseen circle-drawing task and 99.5% of 8,947 poses commanded by deployed policies. A position-only reach map misses every unreachable frame (279/279 and 153/153): reachability is an orientation problem.

Training-set reachability. Anchored at the deployment start pose, 83.3% of the 15,912 training frames are reachable (full IK agrees on 499 of 500 sampled frames), and 34 of 45 episodes contain unreachable frames. The misses cluster at the grasp: 32% and 24% of frames in an episode's middle quarters, against 6% at either end, at a median height of 7 cm against 18 cm. The hand grasps near

the table at orientations the YAM wrist cannot take; at deployment our controller already tilts the tool down by up to 35° when a low target would straighten the elbow.

Task-aware retargeting. A task fixes only part of the pose: a parallel jaw can flip, a marker can roll, a grasp tolerates some tilt. For each frame T_t and allowed rotation O_k we score $\log p(T_t O_k)$ with the MLP and choose a path by Viterbi, with a turning cost ($\leq$15° per frame) and a pull toward the demonstrated pose. All four audit episodes with unreachable frames improve (to 95.9–100% reachable, confirmed by full IK and the deployment follower) in 11–97 ms each. On the training set a 10° tilt alone raises reachability from 83.3% to 93.2%, the same as tilt plus flip. Because the camera rides on the tool, a $\leq$10° tilt barely changes the view, so labels can be retargeted without re-rendering images, whereas filtering at 95% would discard 32 of 45 episodes. Training on retargeted labels is the next experiment.

6 Limitations and conclusion

The T265 is discontinued [14], so units are reseller stock. Policies cover one task with 10 trials per model; we have not compared against a different camera or against YAM leader-arm teleoperation. The audit is kinematic (no collisions), and task freedoms are set by hand. Still, a YAM rig already specifies most of a handheld interface: YAMI records demonstrations in the arm's camera and tool frame without SLAM, they train a policy that completes the task in 6 of 10 trials, and the arm's own model shows where the hand went beyond it.

References

[1] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. In *Robotics: Science and Systems (RSS)*, 2024.

[2] Zhaxizhuoma et al. FastUMI: A scalable and hardware-independent universal manipulation interface. *arXiv preprint arXiv:2409.19499*, 2024.

[3] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[4] A. Allshire et al. Scalable behavior cloning with open data, training, and evaluation, 2026. Hardware list: https://abc.bot/hardware.html.

[5] Allen Institute for AI. MolmoAct 2: An open foundation for robots that work in the real world. https://allenai.org/blog/molmoact2, 2026.

[6] Y. Jiang et al. RoboTTT: Context scaling for robot policies, 2026.

[7] I2RT Robotics. ABC Box. https://i2rt.com/products/abc-box, 2026.

[8] Y. Zhang et al. VOMMI: Collecting and leveraging portable demonstrations for mobile manipulation, 2026. Table I, estimated acquisition cost.

[9] SRA VJTI. Karma: Robot control stack for bimanual YAM and SO100/SO101 arms. https://github.com/SRA-VJTI/karma, 2026.

[10] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

[11] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[12] K. Zakka. mink: Python inverse kinematics based on MuJoCo. https://github.com/kevinzakka/mink, 2024.

[13] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li. On the continuity of rotation representations in neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[14] Intel. Product change notification 807360-01: Discontinuance of RealSense T265, 2023.